UDESC



Rust

Cristiano Damiani Vasconcellos
cristiano.vasconcellos@udesc.br

UDESC

Panic

Uma situação anormal, inesperada, que deve causar o encerramento do programa, por exemplo:

- Acesso a uma posição inválida de um array;
- Divisão por zero;
- Falha em uma asserção;
- Chamar expect() em um dado do tipo Result que é um Err.

```
assert!(a >= 0, "a não pode ser negativo")
```

UDESC

Panic

unwinding (liberação da pilha) é o processo de percorrer a pilha de chamadas em ordem inversa, removendo os frames ativos e executando os destrutores (drop) dos dados que saem de escopo, geralmente após um *panic*.

TOML

```
[profile.release]
panic = "unwind"
```

abort é o encerramento imediato do processo quando ocorre uma falha, sem realizar a liberação da pilha (*unwinding*) e, consequentemente, sem executar os destrutores (drop) dos objetos que ainda estão ativos.

TOML

```
[profile.release]
panic = “abort"
```

UDESC

Result

Rust não possui suporte a exceções, no lugar as funções que querem sinalizar uma condição de erro para quem as chamou o fazem por meio do retorno de um dado do tipo Result:

```
enum Result<T, E> {
   Ok(T),
   Err(E),
}
```

Funções devem retornar Result quando erros são esperados e podem ser recuperados, por exemplo, funções que executam entrada e saída de dados.

UDESC

Result

```
use std::fs;
fn main() {
    let conteudo = fs::read_to_string("/Temp/teste.txt")
    .expect("Não foi possível abrir o arquivo");

    println!("\n{conteudo}\n");
}
```

UDESC

Result

```
use std::fs;
fn main() {
    let r = fs::read_to_string("/Temp/teste.txt");
    match r {
        Ok (conteudo) => {println!("\n{conteudo}\n");}
        Err (_)       => {println!("Erro na abertura do arquivo");}
    }
}
```

UDESC

Result

```
use std::fs;
fn main() {
    match imprime("/Temp/teste.txt") {
        Ok (tam) => {println!("\nO arquivo possui {tam} caracteres\n");}
        Err (_)     => {println!("Erro na abertura do arquivo");}
    }
}

fn imprime (arq:&str) -> Result <usize, std::io::Error> {
    match fs::read_to_string(arq) {
        Ok (conteudo) => {
            println!("{conteudo}");
            Ok (conteudo.len())
        }
        Err (e) => Err (e)
    }
}
```

UDESC

Propagação do Erro

Em muitas situações queremos executar algo que pode ocasionar em um erro, mas não queremos tratar o erro nesse lugar, para esses casos Rust possui o operador **?** que propaga o erro para o ponto de chamada da função.

UDESC

Result

```
use std::fs;
fn main() {
    match imprime("/Temp/teste.txt") {
        Ok (tam) => {println!("\nO arquivo possui {tam} caracteres\n");}
        Err (_)     => {println!("Erro na abertura do arquivo");}
    }
}

fn imprime (arq:&str) -> Result <usize, std::io::Error> {
    let conteudo = fs::read_to_string(arq)?;

    println!("{conteudo}");
    Ok (conteudo.len())
}
```